

Gestionnaire de Mémoire (Jour 6)

1. Gestionnaire de Mémoire Physique (PMM)

Définition

Le PMM est la couche la plus basse de la gestion mémoire. Son rôle est de gérer la RAM réelle (physique) par blocs de **4 Ko** (appelés "Pages" ou "Frames").

Pourquoi est-ce nécessaire ?

Sans PMM, le noyau ignore quelles zones de la RAM sont libres. Si on veut charger un nouveau programme ou créer une structure de données, on risque d'écraser :

- Le code du noyau lui-même.
- La mémoire vidéo (VGA).
- Des zones réservées par le BIOS/UEFI.

Choix de Conception : Bitmap vs Stack

- **Bitmap (Bimap)** : Un tableau de bits où chaque bit représente une page physique (1 = utilisée, 0 = libre).
 - *Avantage* : Très simple, permet de trouver des blocs de pages contiguës facilement.
 - *Inconvénient* : La recherche d'une page libre peut être lente si la mémoire est grande ($O(n)$).
- **Stack (Pile de pages libres)** : Une liste (ou pile) contenant les adresses des pages libres.
 - *Avantage* : Allocation/Libération ultra-rapide ($O(1)$).
 - *Inconvénient* : Difficile d'allouer plusieurs pages contiguës.

Nous utiliserons probablement un **Bitmap** pour sa simplicité et sa capacité à gérer la fragmentation au début.

2. Gestionnaire de Mémoire Noyau (Kalloc / kmalloc)

Définition

Le `kmalloc()` est l'équivalent du `malloc()` du C standard, mais pour le noyau. Il permet d'allouer de petites quantités de mémoire (ex: 32 octets pour une structure Task).

Pourquoi est-ce nécessaire ?

Le PMM ne donne que des blocs de 4096 octets. Si vous avez besoin de seulement 20 octets, gaspiller 4096 octets est inefficace (**Fragmentation interne**). Le `kmalloc` découpe les pages fournies par le PMM en petits morceaux.

Fonctionnement simplifié

1. Le kmalloc demande une page au PMM.
2. Il divise cette page en "chunks".
3. Il maintient une liste chaînée des chunks libres.

3. Focus Sécurité : Heap Overflow (Débordement de Tas)

Comment ça arrive dès la conception ?

Dans un allocateur simple, on stocke souvent les métadonnées (taille du bloc, pointeur vers le bloc suivant) juste **avant** les données retournées à l'utilisateur.

```
[ METADATA (Taille=16) ] [ DONNÉES UTILISATEUR (16 octets) ] [ METADATA (Prochain bloc) ]
```

Si l'utilisateur écrit 20 octets dans un bloc de 16, il écrase les **METADATA** du bloc suivant.

Conséquences

- **Corruption du système** : Au prochain free() ou malloc(), le noyau utilise des métadonnées corrompues, ce qui peut provoquer un crash (Kernel Panic).
- **Exploitation** : Un attaquant peut manipuler ces métadonnées pour forcer kmalloc à retourner un pointeur vers une zone sensible (ex: la GDT ou le code du noyau), permettant de modifier le comportement de l'OS.

Physical Memory Manager (PMM)

Le PMM utilisera un **Bitmap** pour suivre l'état de chaque page de 4 Ko.

- Nous gérerons jusqu'à 128 Mo de RAM (ajustable).
- Pour 128 Mo, le bitmap fera 4 Ko (1 page).
- Les pages occupées par le noyau (1 Mo + taille du noyau) seront marquées comme réservées au démarrage.

Plan de Vérification

Tests Automatisés

- Test d'allocation de 1000 pages et vérification de la cohérence du bitmap.
- Test de kmalloc avec différentes tailles (petites, grandes, multi-pages).

Vérification Manuelle

- Affichage de l'état de la mémoire avant/après allocation.
- Simulation d'un dépassement de tampon pour observer le crash/comportement.

```
one@DESKTOP-5RV2RM1:/mnt/c/Users/MOH/Documents/Lab1_CYBER$ tree 6_ProjetOS/
6_ProjetOS/
├── framebuffer.c
├── framebuffer.h
├── gdt.c
├── gdt.h
├── gdt_flush.s
├── idt.c
├── idt.h
├── interrupts.s
├── io.h
├── io.s
├── iso
│   └── boot
│       ├── grub
│       │   ├── menu.lst
│       │   └── stage2_eltorito
│       └── kernel.elf
├── kalloc.c
├── kalloc.h
├── kmain.c
├── launcher.sh
├── link.ld
├── loader.s
├── os.iso
├── paging.c
├── paging.h
├── paging_asm.s
├── pmm.c
└── pmm.h

4 directories, 25 files
```

Puis on passe à la compilation et à la création de la vm.

Machine View

```

--- INITIALISATION MEMOIRE ---
PMM Initialise. Pages libres: 32500
Kmalloc Initialise.

--- TEST PMM ---
Page 1 allouee a: 0x0010D000
Page 2 allouee a: 0x0010E000
Pages libres: 32497
Page 3 (apres free P1) allouee a: 0x0010D000

--- TEST KMALLOC ---
S1 alloue a: 0x0010C010
S2 alloue a: 0x0010C030
Libération S1...
Test de corruption (Simulation overflow)...
S3 alloue a: 0x0010C010
Corruption du bloc suivant...
Tentative de free du bloc corrompu (check headers)...

```

Mais une question se pose, pourquoi avoir allouer l'adresse 0x00000000 n'est pas allouée ?

C'est grâce au pmm_init qui par mesure de sécurité se doit de marquer comme "utilise" les adresses de 0x00000000 jusqu'à la fin de la taille du kernel pour éviter que ces adresses sont réallouée pour écrire autre chose dessus.

```

void pmm_init(unsigned int kernel_end) {
    // 1. Initialiser tout le bitmap à 0 (toutes les pages libres)
    for (int i = 0; i < BITMAP_SIZE; i++) {
        bitmap[i] = 0;
    }
    free_pages = TOTAL_PAGES;

    // 2. Réserver les pages de 0 à la fin du noyau
    // On arrondit kernel_end à la page supérieure
    unsigned int pages_to_reserve = (kernel_end + PAGE_SIZE - 1) / PAGE_SIZE;

    for (unsigned int i = 0; i < pages_to_reserve; i++) {
        pmm_set_page(i);
    }
}

```

```
void* pmm_alloc_page() {
```

```
^G Help
^X Exit
```

```
^O Write Out
^R Read File
```

```
^W Where Is
^N Replace
```

```
^K Cut
^U Paste
```

```
^T Execute
^J Justify
```

```
^C Location
^_ Go To Line
```

```
M-U Undo
M-E Redo
```

```
M-A Set Mark
M-6 Copy
```

Si on l'enlève :

```
QEMU - Press Ctrl+Alt+G to release grab x
Machine View
--- INITIALISATION MEMOIRE ---
PMM Initialise. Pages libres: 32768
Kmalloc Initialise.

--- TEST PMM ---
Page 1 allouee a: 0x00001000
Page 2 allouee a: 0x00002000
Pages libres: 32765
Page 3 (apres free P1) allouee a: 0x00001000

--- TEST KMALLOC ---
S1 alloue a: 0x00000000
S2 alloue a: 0x00000000
Libération S1...
Test de corruption (Simulation overflow)...
S3 alloue a: 0x00000000
Corruption du bloc suivant...
Tentative de free du bloc corrompu (check headers)...
```

Mais c'est dangereux car là on a écrit des choses dans une zone sensible de la RAM. Si notre

kmalloc mal configuré (sans PMM) renvoie une adresse qui tape pile sur la GDT ou l'IDT de l'OS, au prochain appel système ou interruption... **l'OS crache et la vm reboot immédiatement !**

D'ailleurs on peut faire un test. Supprimer l'initialisation de PMM et ajouter ce code C puis lancer le dans le kmain , l'OS plantera a cout sûr



```
void test_idt_destruction() {
    fb_print("\n--- TEST: ECRASSEMENT DE L'IDT ---\n", 12); // Rouge

    // L'IDT est chargée quelque part en mémoire.
    // Pour l'expérience, on va brutalement écrire des zéros là où on pense qu'elle est.
    // Ou mieux: on va écrire des zéros à l'adresse 0, et simuler que l'IDT était là (ce qui n'est pas le cas ici, mais le principe reste).

    // Pour garantir le crash, on va écraser la structure de l>IDTR (le registre qui pointe vers l'IDT).
    struct idt_ptr {
        unsigned short limit;
        unsigned int base;
    } __attribute__((packed));

    struct idt_ptr idtr;
    asm("sidt %0" : "=m"(idtr)); // On récupère l'adresse réelle de l'IDT

    fb_print("IDT est a: ", 15);
    fb_print_hex(idtr.base);

    fb_print("\nOn efface l'IDT ... \n", 4);

    // On efface l'IDT avec des zéros
    unsigned char* ptr = (unsigned char*)idtr.base;
    for(int i = 0; i < 256 * 8; i++) {
        ptr[i] = 0;
    }

    fb_print("Generation d'une interruption...\n", 15);
    // Le CPU va chercher le gestionnaire dans l'IDT... qui est vide ! -> CRASH
    asm volatile("int $0x3");
}
```